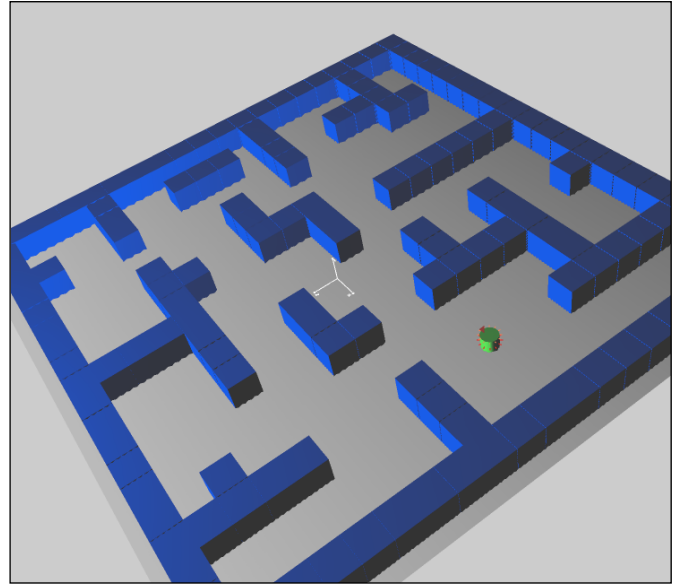


Evolutionary Robotics

Nicolas Bredeche

Sorbonne Université
ISIR, UMR 7222
Paris, France
nicolas.bredeche@sorbonne-universite.fr



UE AROB - Apprentissage et Robotique
M1 informatique, parcours Androïde

rev. 2026-02-27



Outline

2

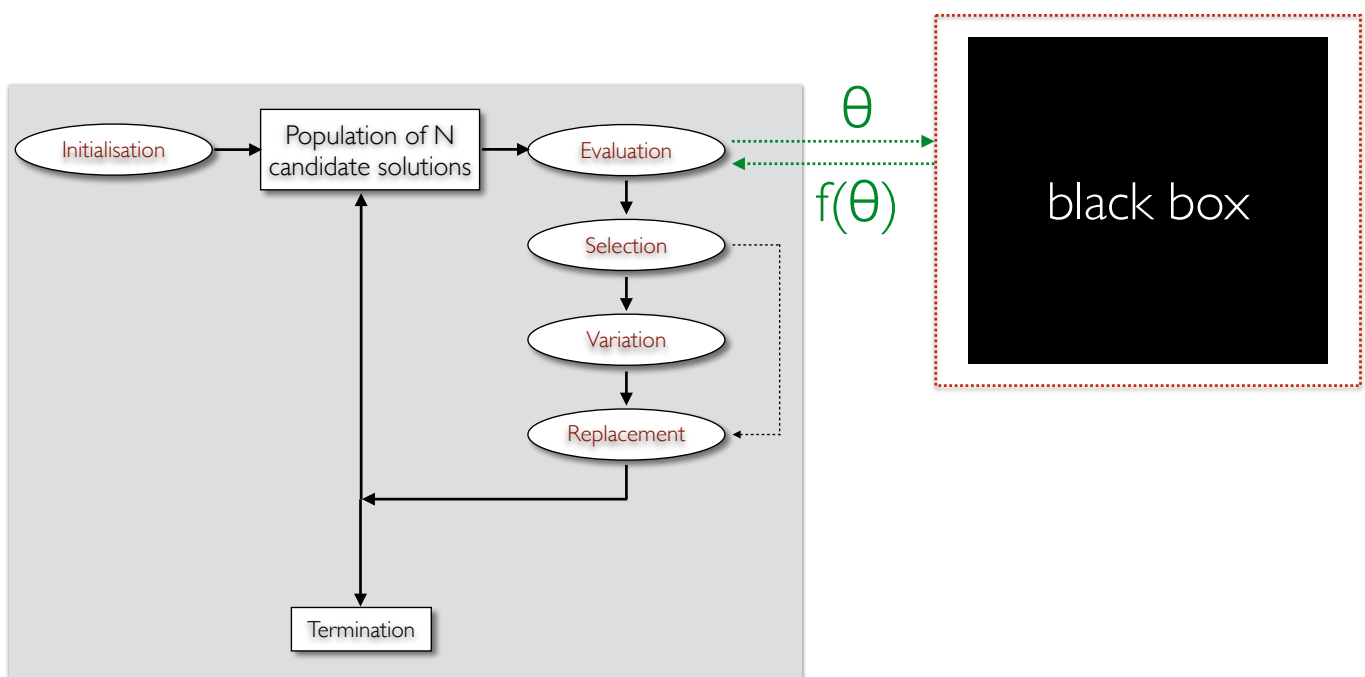
- Evolutionary robotics
 - ▶ ES as reinforcement learning method: direct policy search
 - ▶ Some examples of problems solved with evolutionary robotics
 - ▶ Inside the blackbox — concepts and definitions

Evolutionary Robotics

principles and examples

Evolutionary Robotics

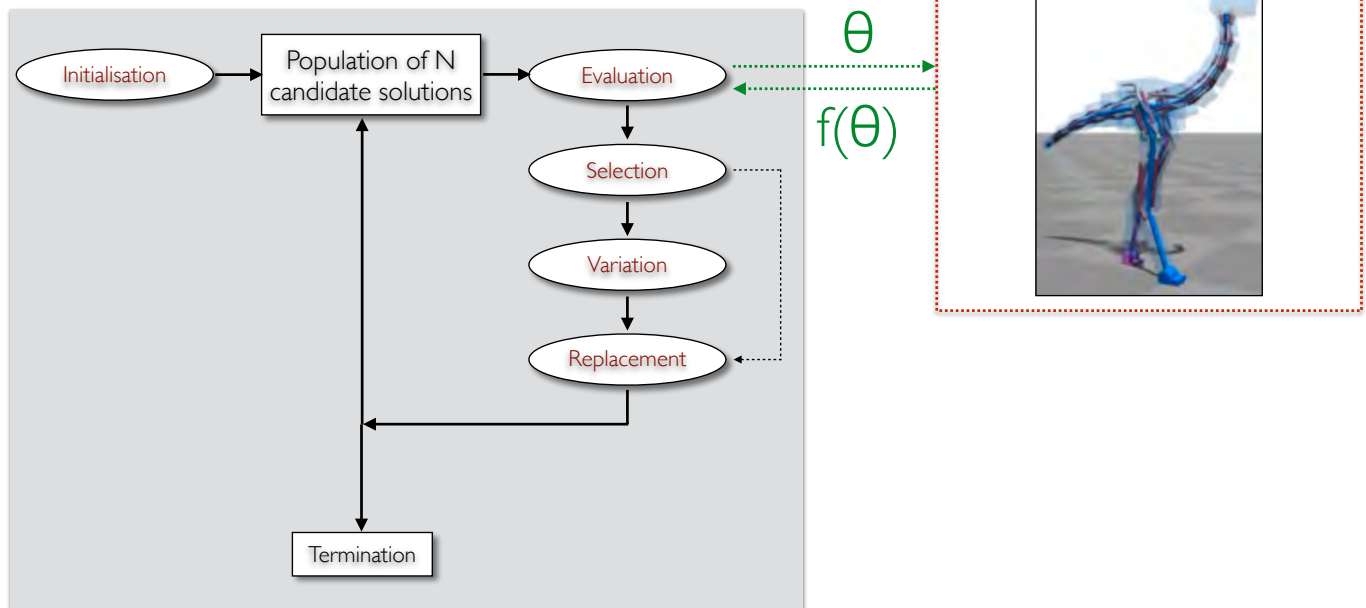
4



θ : parameters

f : objective function

$f(\theta)$: fitness

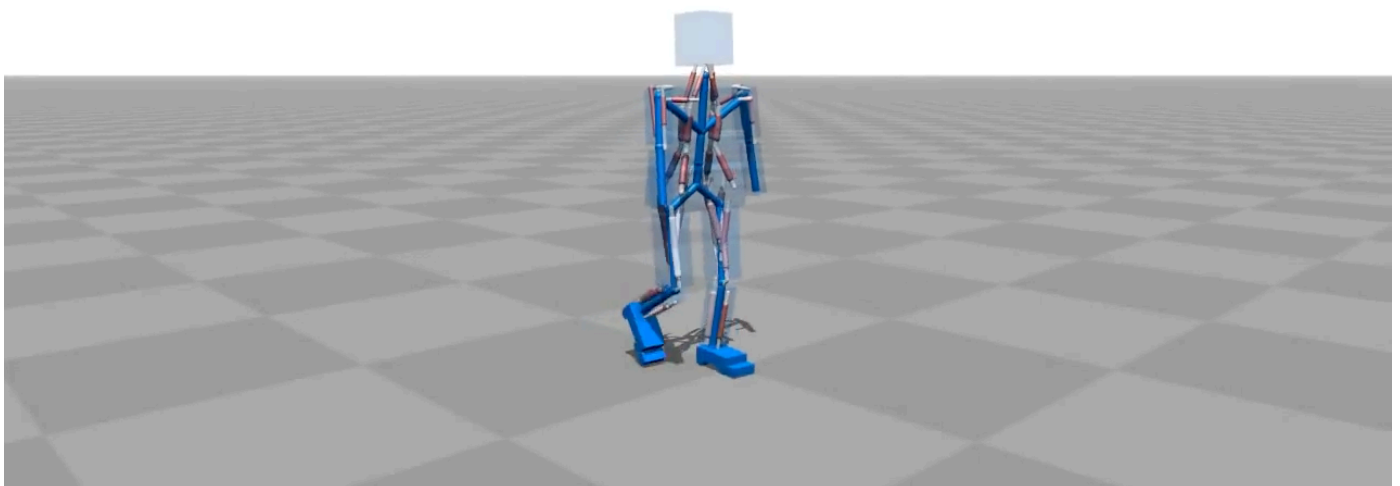


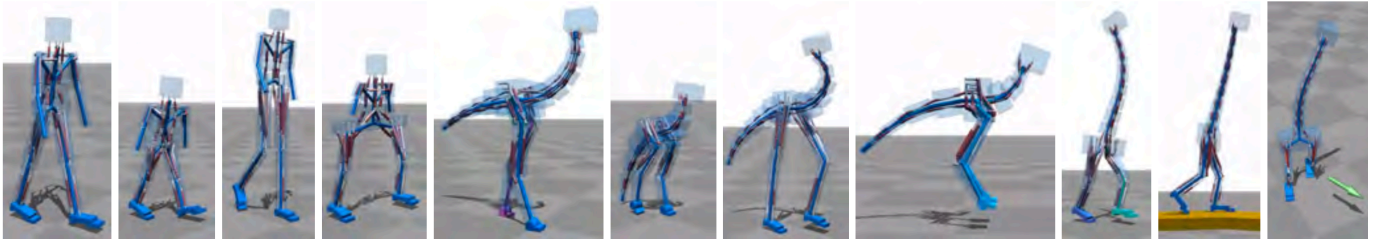
θ : parameters

f : objective function

$f(\theta)$: fitness

Geijtenbeek et al. (2013) "Flexible muscle-based locomotion for bipedal creatures", SIGGRAPH ASIA 2013





Geijtenbeek et al. (2013) "Flexible muscle-based locomotion for bipedal creatures", SIGGRAPH ASIA 2013

Target features for locomotion

8

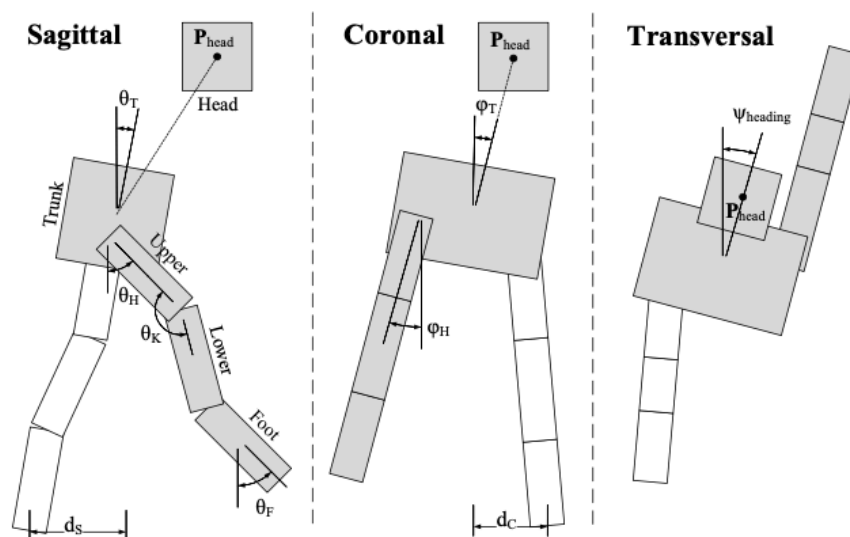


Figure 6: Target features for trunk, head and swing leg, shown in sagittal, coronal and transversal projection.

→ objective function: match the desired gait (pose, orientation and velocity)

$$\text{minimise}(\bar{E}(\mathbf{K})) = \bar{E}_{\text{speed}} + \bar{E}_{\text{headori}} + \bar{E}_{\text{headvel}} + \bar{E}_{\text{slide}} + \bar{E}_{\text{effort}}$$

1. muscle attachment points

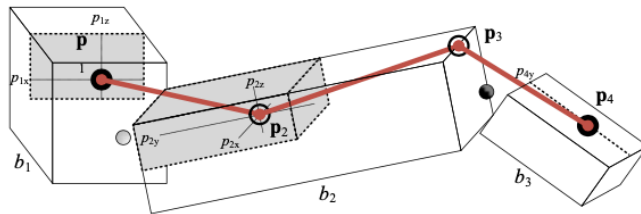
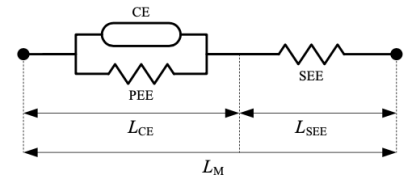


Figure 3: Muscle attachment points that will be optimized within a constrained region. In this example, muscle point $\mathbf{p}_1$ is constrained to a 2D surface, muscle point $\mathbf{p}_2$ is constrained to a 3D volume, $\mathbf{p}_3$ is fixed, and $\mathbf{p}_4$ is constrained to a line. The actual areas used in our experiments are shown in Figure 8.



Three components of a Hill-type muscle

- Contractile Element (muscle fibers)
- Parallel Elastic Element (elastic material surrounding muscle fibers)
- SEE: serial elastic element (tendons)

Muscle is controlled by **neural excitation** signal u to each muscle, which is output by the control system

2. muscle-based feature control

Given:

- current state and desired state

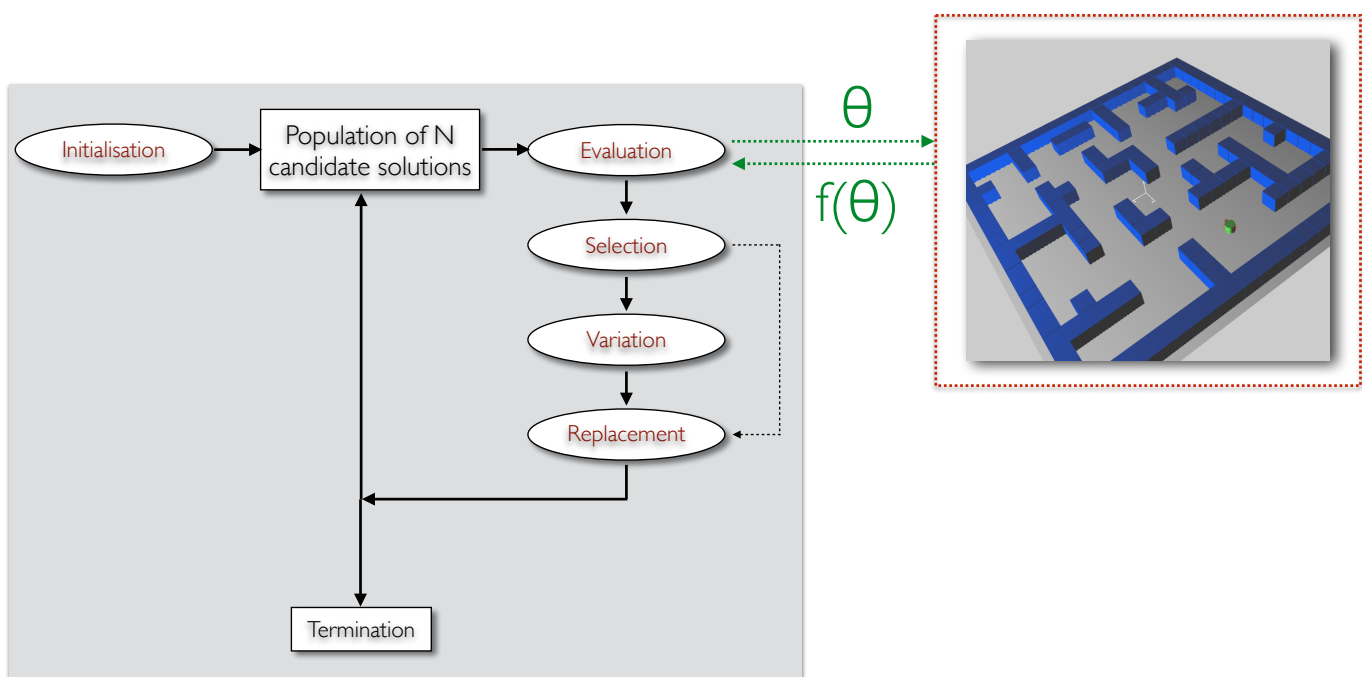
Minimise the state difference

- feedback force and feedback torque through Proportional-Derivative control

→ CMAES for (1) muscle attachment points (2) control parameters

Geijtenbeek et al. (2013) "Flexible muscle-based locomotion for bipedal creatures", SIGGRAPH ASIA 2013

Evolutionary Robotics — robot navigation

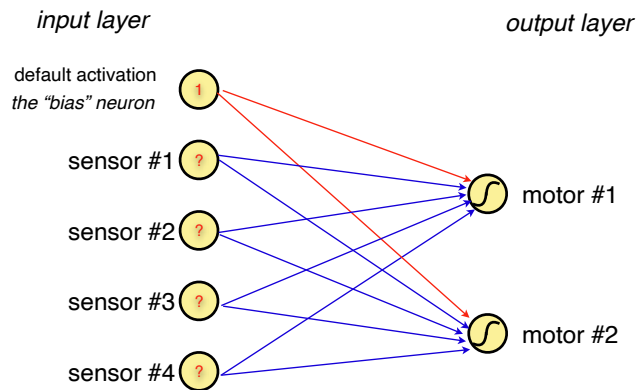
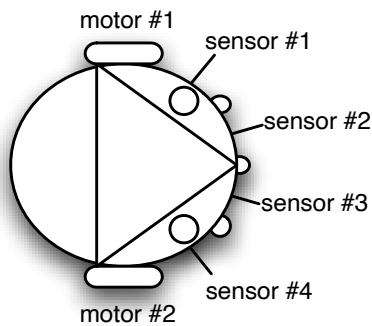


θ : parameters

f : objective function

$f(\theta)$: fitness

Blackbox optimisation and robotics



NN weights as parameters in $\mathbb{R}^{10}$

$$output_j = \tanh\left(\sum_i w_{i,j} * input_i\right)$$

$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

with x , the neural network weights

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

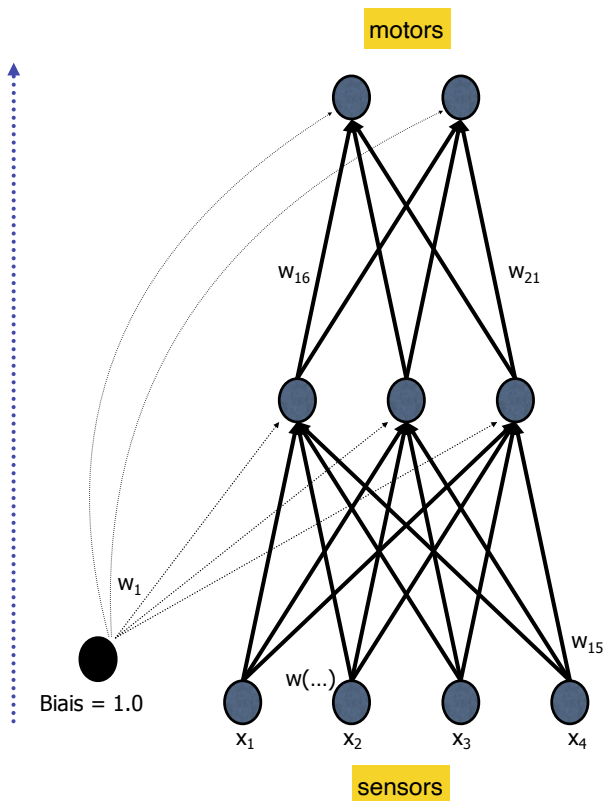
the best trajectory is obtained w/ parameters x^*

Neural networks for robot control

primer / reminder

Multi-Layered Perceptron (MLP)

13



$$a_i = \sum_{j=0}^n w_{ji} a_j$$

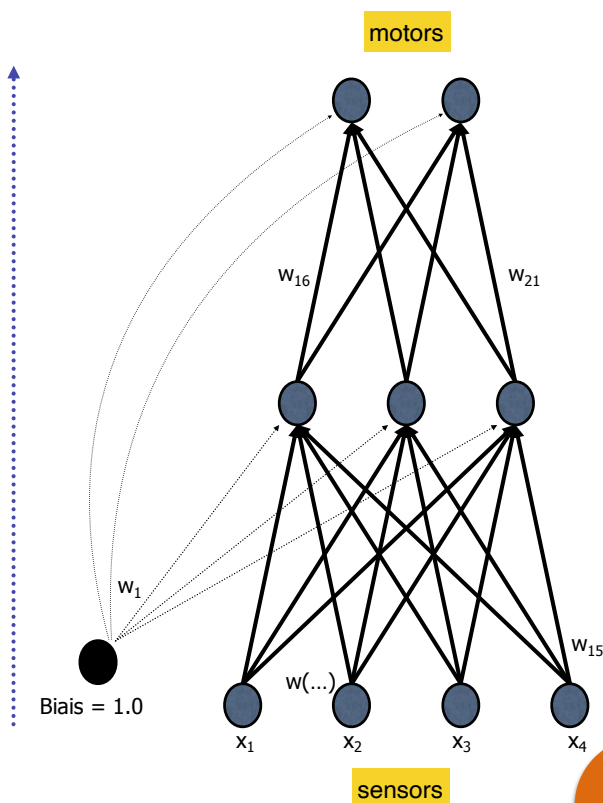
linear activation function

$$a_j = f_{activation}\left(\sum_{k=0}^n w_{kj} x_k\right)$$

$$a_k = (\text{inputs})$$

Signal propagation in a MLP

14



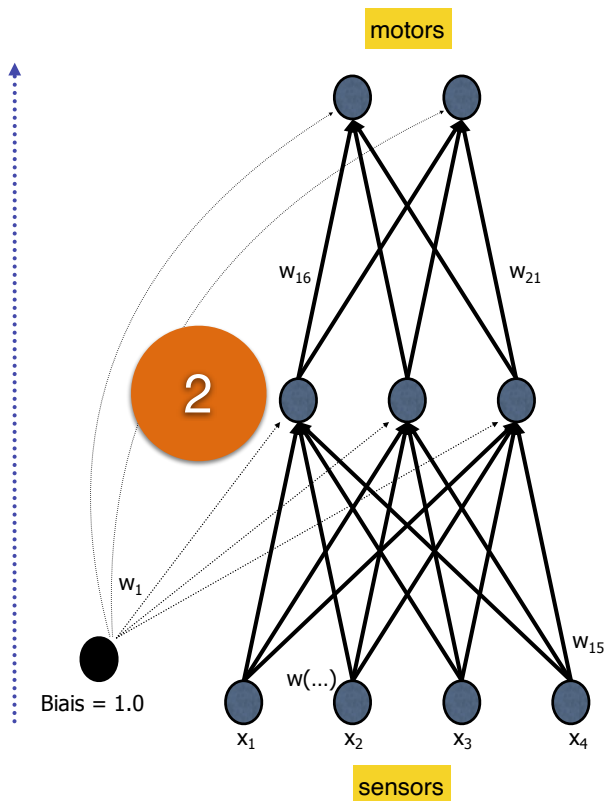
$$a_i = \sum_{j=0}^n w_{ji} a_j$$

$$a_j = f_{activation}\left(\sum_{k=0}^n w_{kj} x_k\right)$$

$$a_k = (\text{inputs})$$

Signal propagation in a MLP

15



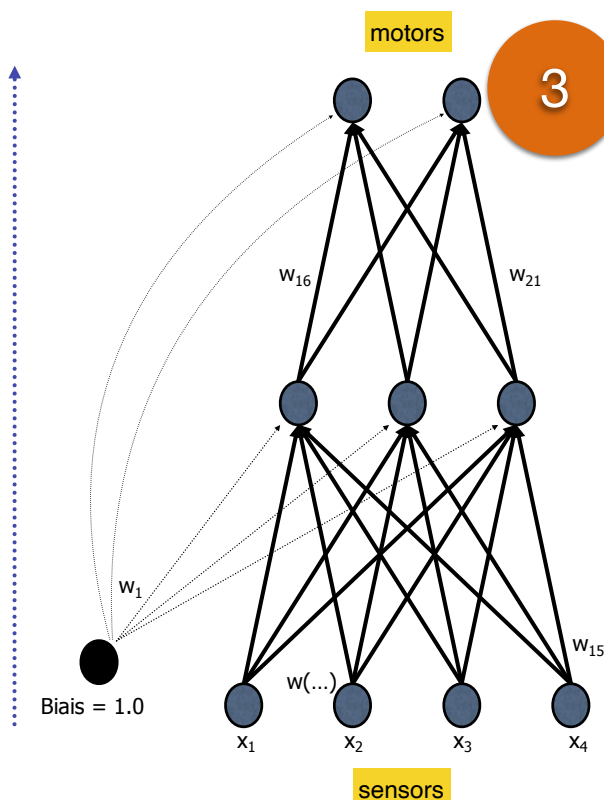
$$a_i = \sum_{j=0}^n w_{ji} a_j$$

$$a_j = f_{activation} \left(\sum_{k=0}^n w_{kj} x_k \right)$$

$$a_k = (inputs)$$

Signal propagation in a MLP

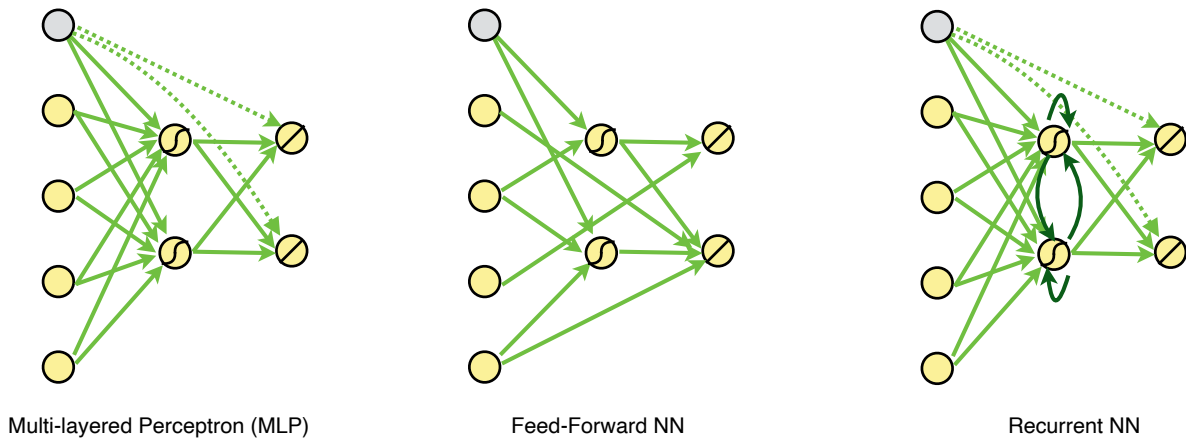
16



$$a_i = \sum_{j=0}^n w_{ji} a_j$$

$$a_j = f_{activation} \left(\sum_{k=0}^n w_{kj} x_k \right)$$

$$a_k = (inputs)$$

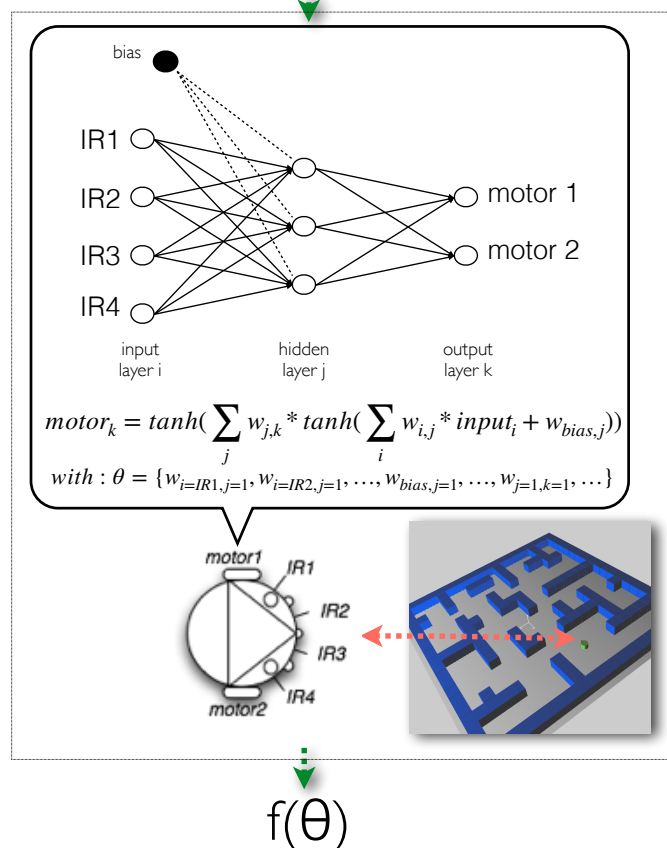


Also...

- Neuron type: discrete, continuous, time-delayed, etc.
- Update rule: additive, multiplicative, etc.
- Activation function: linear, Heaviside, sigmoid, tanh, radial basis function, etc.

Genotype-to-phenotype mapping

neural net weights θ



$$\Pi_{\theta} : S \rightarrow A$$

$$J(\theta) : \mathbb{R}^n \rightarrow \mathbb{R}$$

$$\theta^* = \operatorname{argopt}_{\theta \in \Theta} J(\theta)$$

- ES as a reinforcement learning method
 - ▶ Direct **policy** search: find the best θ w.r.t. the objective function
 - ▶ Viewpoint from ES: just another **blackbox optimisation problem**

θ : parameters (e.g. neural net weights, ODE parameter values, etc.)

$J(\theta)$: “return” (sum of immediate rewards) — J is similar to a performance function

QUIZZ

Q: l'algorithme (I+I)-ES peut-être utiliser pour optimiser...

1. le profil du réacteur d'une fusée
2. les poids d'un réseaux de neurones contrôlant un robot mobile
3. les fréquences d'oscillation et de déphasage entre jointures d'un robot articulé?

Q: dans la question précédente, peut-on utiliser le *même* algorithme pour les trois tâches?

1. oui
2. non

Q: soit un contrôleur basé sur un MLP, on peut optimiser...

1. les poids des arcs
2. la pente de la fonction d'activation sigmoïde
3. la structure (topologique) du réseaux de neurones

Q: dans la question précédente, peut-on utiliser le *même* algorithme pour les trois tâches?

1. oui
2. non

In the following:

we use a classic ANN controller for navigation tasks
to present the basic concepts of evolutionary robotics

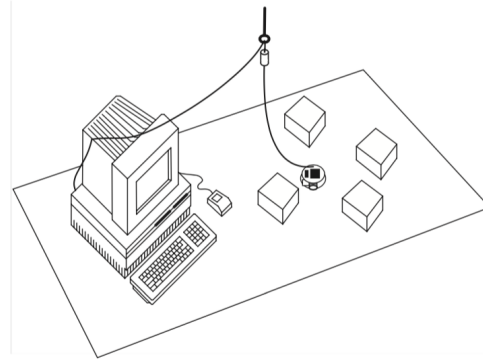
Results shown hereafter can be generalised to other domain and tasks.

e.g.: a sequence of actions may influence...

- the trajectory of a mobile robot in an arena
- the trajectory of the hand of a robotic arm in a 3D space
- the trajectory of a wheeled robot balancing an upside-down pendulum
- the trajectory in the joint space of a running multi-legged robot

Many concepts presented in this course are taken from Nolfi and Floreano's 2000 reference book on evol. robotics

Sensori-motor loop



$$fitness(x) = \int_t^T V_{translation}^t * (1 - V_{rotation}^t) * (sensor_{min})$$

$V_{translation}$: translational speed

$V_{rotation}$: rotational speed

$sensor_{min}$: value of the *most* stimulated sensor

if no obstacle: returns 1

otherwise: returns a value in $[0,1[$

Notes:

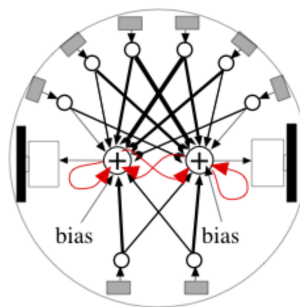
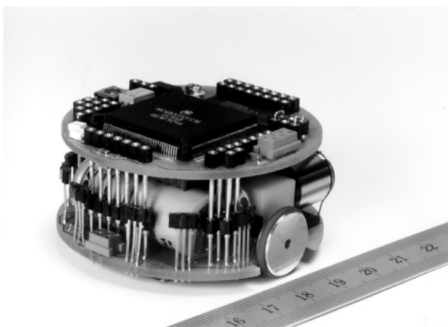
each term is normalized between 0 and 1

the rotational speed is given in absolute value

[Floreano, Mondada 1994][Nolfi, Floreano, 2000]

Experimental setup

28



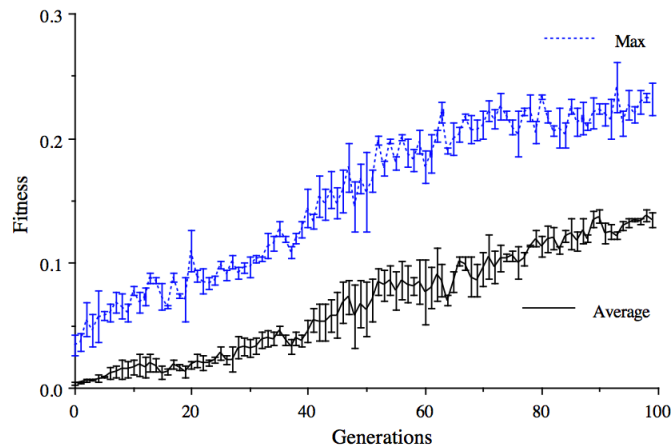
Population size	80
Generation number	100
Crossover probability	0.1
Mutation probability	0.2
Mutation range	± 0.5
Initial weight range	± 0.5
Final weight range	Not bounded
Life length	80 actions
Action duration	300 ms

• experimental conditions

- Artificial NN sigmoid function, recurrent NN, genome: weights and activation
- Naive evolutionary algorithm basic mutation/crossover and selection operators
- Real robot 40 min per generation; 100 generations of 80 indiv.; total duration: 66 hours (!)

[Floreano, Mondada 1994][Nolfi, Floreano, 2000]

Average and best individual fitnesses over generations (3 trials/dot)



- ▶ First generations
 - ▶ spin around and/or crash into walls
- ▶ after 20 generations
 - ▶ most are spinning around, but some start avoiding obstacles
- ▶ after 50 generations
 - ▶ move fast + avoid obstacles + independent of starting point

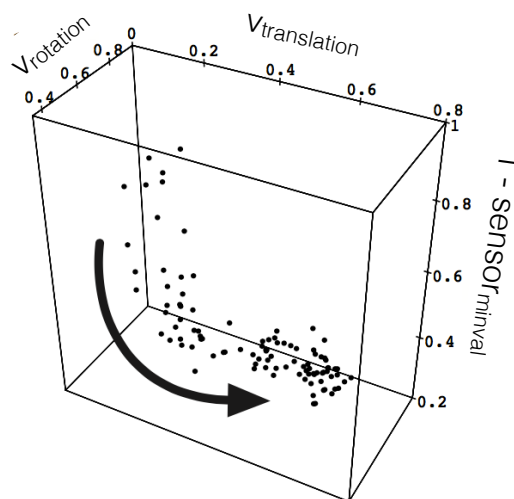
now: prefer "evaluations" over "generations" on x-axis

[Floreano, Mondada 1994][Nolfi, Floreano, 2000]

Trajectory during evolution

30

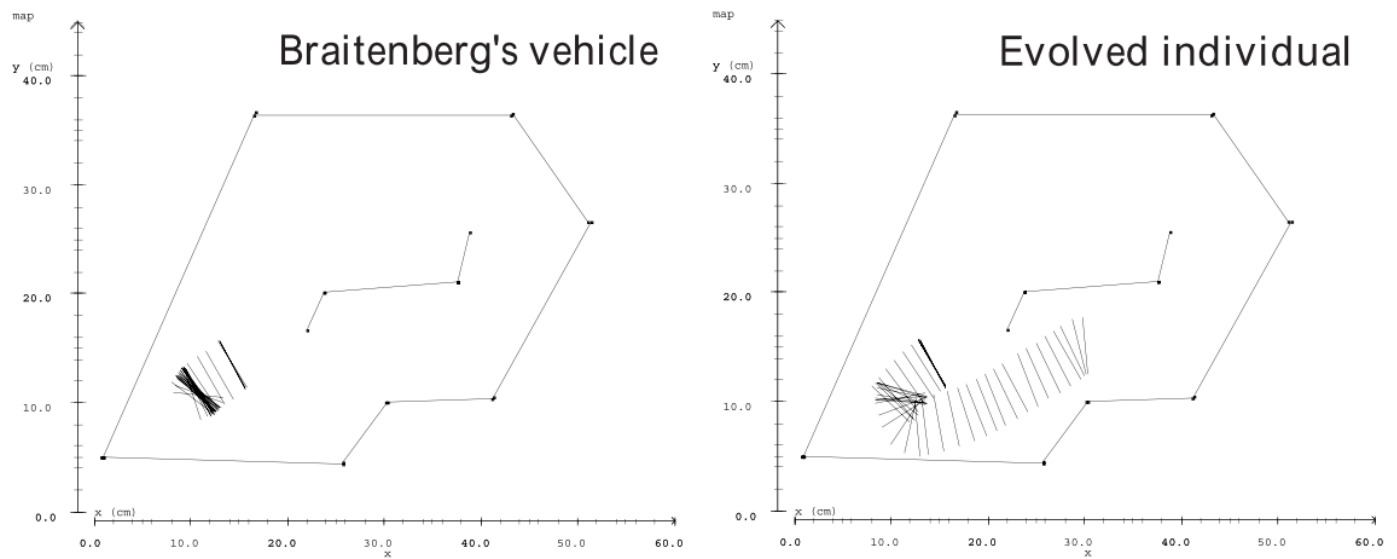
(each dot represents the total fitness of the best individual of a specific generation)



- ▶ First generations
 - ▶ spin around and/or crash into walls
- ▶ after 20 generations
 - ▶ most are spinning around, but some start avoiding obstacles
- ▶ after 50 generations
 - ▶ move fast + avoid obstacles + independent of starting point

illustration from [Floreano, Mondada, 1994], with modified labels

[Floreano, Mondada 1994][Nolfi, Floreano, 2000]



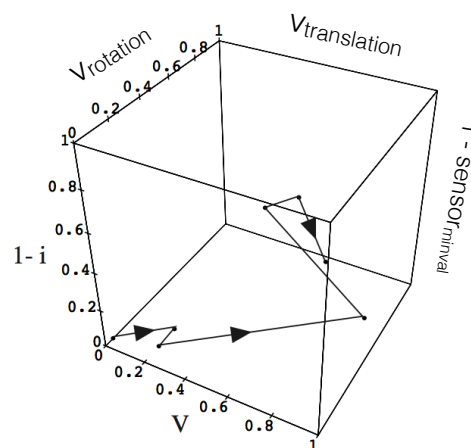
Qualitative comparison

[Floreano, Mondada 1994][Nolfi, Floreano, 2000]

Trajectory of the best individual during its lifetime

32

(instantaneous fitnesses of a specific individual during one evaluation)



Trajectory of a robot in the fitness space

monitoring instantaneous fitness values over time
starting from an unfavourable starting point (i.e. stuck in a corner)

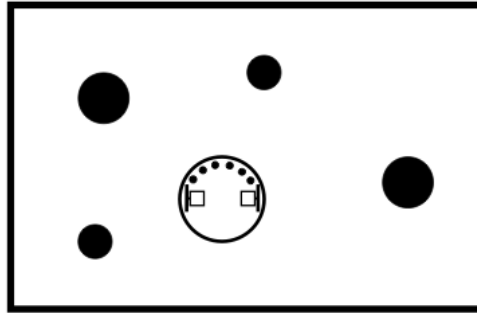
[Floreano, Mondada 1994][Nolfi, Floreano, 2000]

Perceptual aliasing

sensory ambiguities

34

- Definitions
 - Markovian process [Markov, 1906]
 - ▶ a **memory-less** process — e.g. (deterministic) ANN with no recurrence
 - ▶ i.e. the future state depends **only** on the current state
 - Perceptual Aliasing [Whitehead, Ballard, 1991]
 - ▶ two+ situations are perceived as the same
 - ▶ problem: what if each situation requires a different response?



- Task

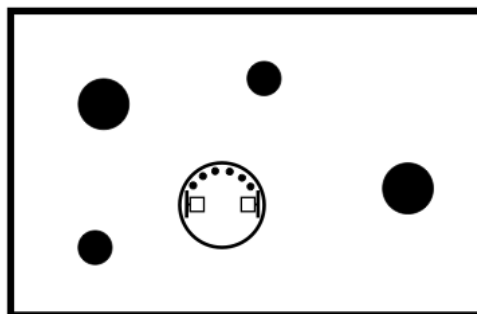
- ▶ get close to the small cylinders
- ▶ move away from the large cylinders
- ▶ avoid the walls

[Nolfi, Floreano, 2000][Nolfi, 2002 (Neurocomputing 42)]

Perceptual aliasing (cont.)

36

supervised
learning



- Offline study : assessing the discrimination ability
 - ▶ Goal: identify ambiguous perception wrt robot relative position
 - Supervised learning task
 - input: sensors
 - label: type of obstacle
 - Method: neural network + back-propagation

[Nolfi, Floreano, 2000][Nolfi, 2002 (Neurocomputing 42)]

supervised
learning

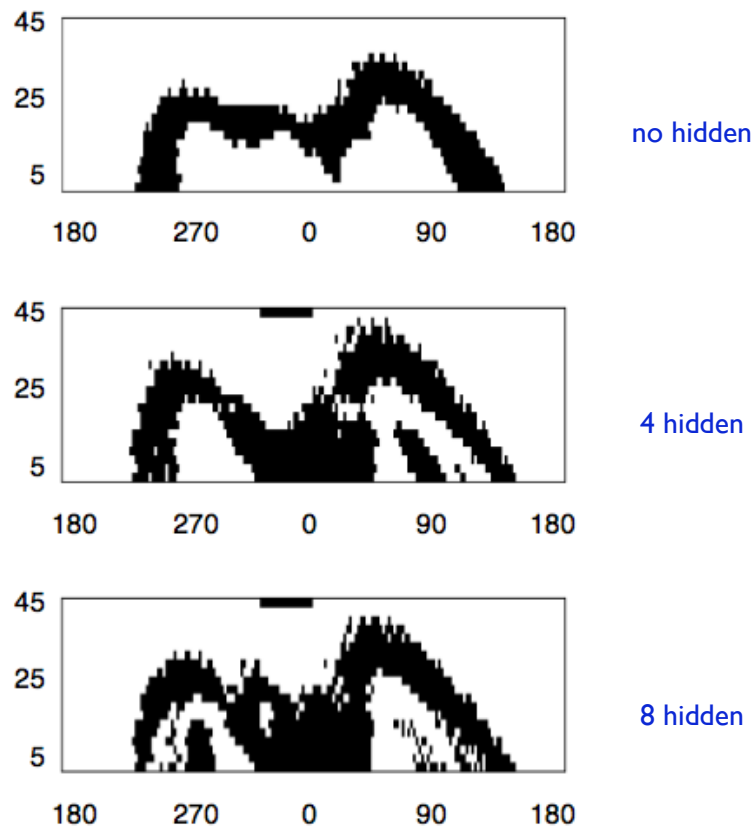


Figure 3. The areas in black represent the relative positions from which networks are able to correctly discriminate the sensory patterns belonging to walls from those belonging to small cylindrical objects. The three pictures (from top to bottom) represent the result for the best simulation with no hidden, 4 hidden, and 8 hidden units, respectively.

[Nolfi, Floreano, 2000][Nolfi, 2002 (Neurocomputing 42)]

offline analysis : big vs. small cylinders

supervised
learning

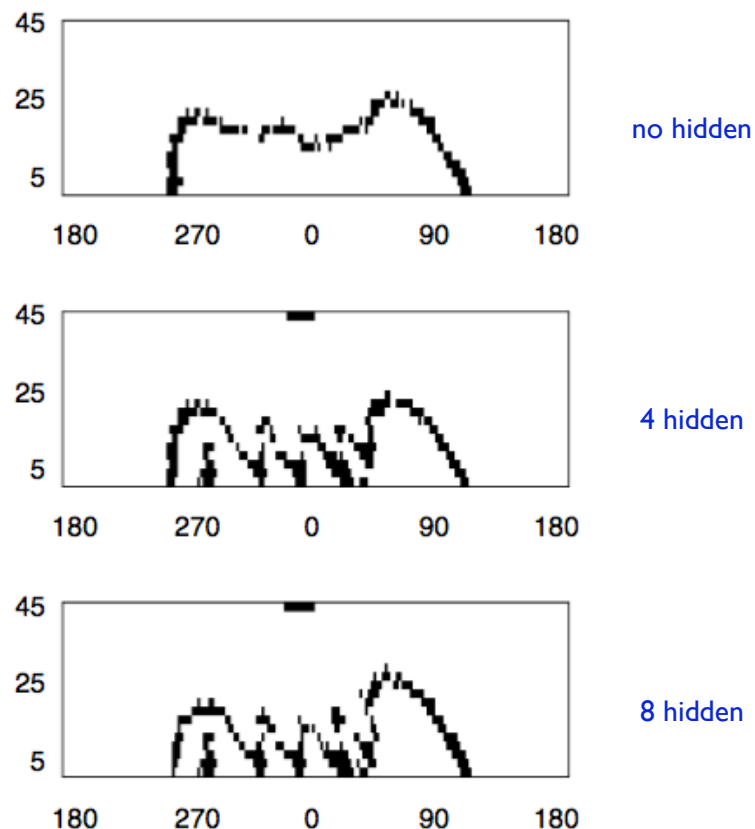


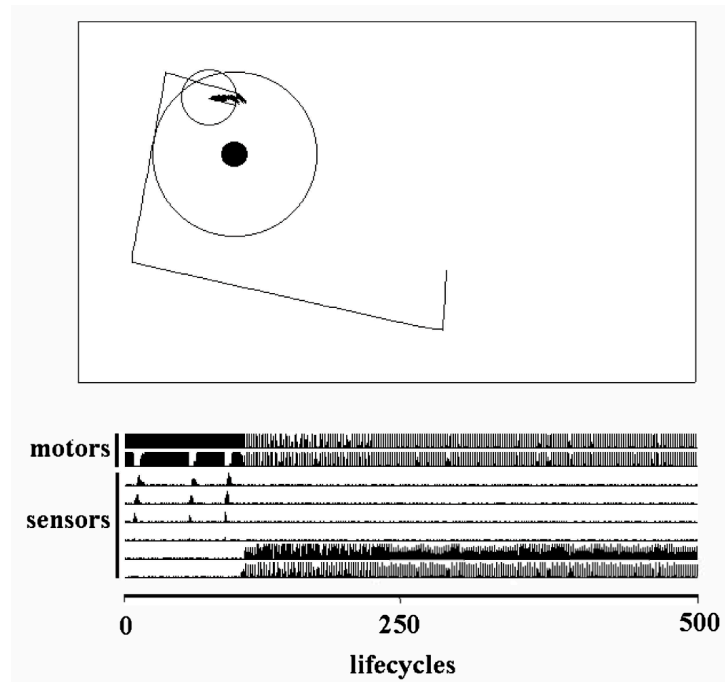
Figure 3. The areas in black represent the relative positions from which networks are able to correctly discriminate the sensory patterns belonging to walls from those belonging to small cylindrical objects. The three pictures (from top to bottom) represent the result for the best simulation with no hidden, 4 hidden, and 8 hidden units, respectively.

[Nolfi, Floreano, 2000][Nolfi, 2002 (Neurocomputing 42)]

evolved
controller

Robot trajectory
in the physical space

Sensorimotor experience
through time



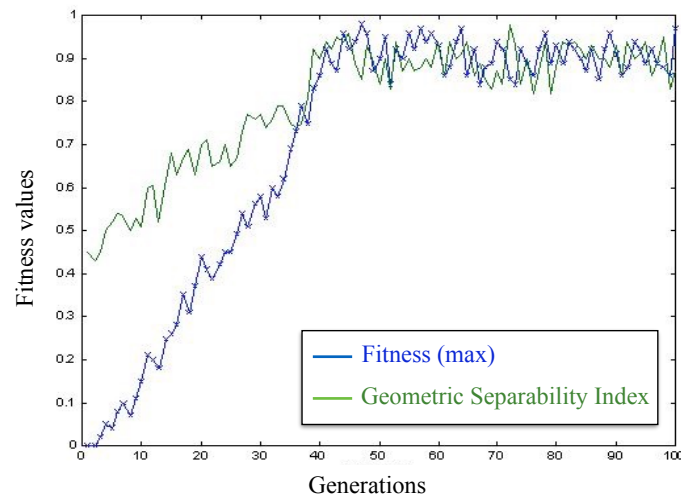
Great results, but how?

[Nolfi, Floreano, 2000][Nolfi, 2002 (Neurocomputing 42)]

Evolution curve

40

evolved
controller



Fitness function

- distance to the small cylinders (minimise, perception threshold)
- distance to large cylinders (maximise, perception threshold)
- avoid the walls

Geometric Separability Index (GSI)

- measure the occurrence of ambiguous situation wrt task
- GSI = 1 means no confusion
- GSI = 0.5 means confusion

evolved
controller

Trajectories (here: four walls and one cylinder)

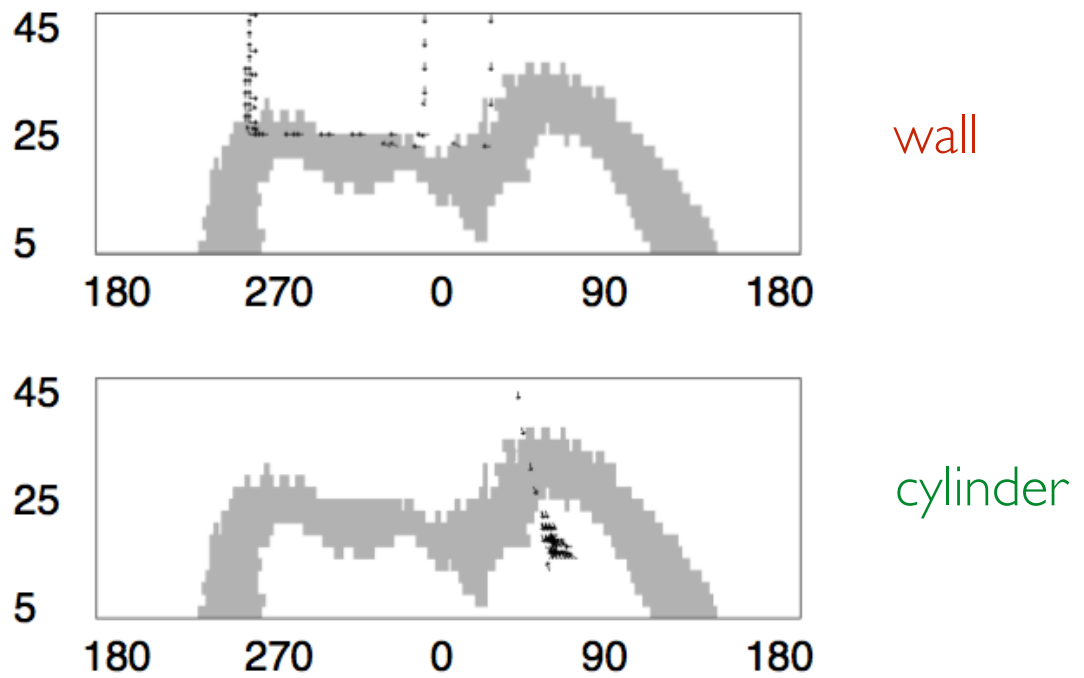


Figure 9. Angular trajectories of an evolved individual close to a wall (top graph) and to a cylinder (bottom graph). The picture was obtained by placing the individual at a random position in the environment, leaving it free to move for 500 cycles, and recording displacements at positions relative to the two types of objects for distances smaller than 45 mm. For sake of clarity, arrows are used to indicate the relative direction but not the amplitude of movements. The gray area represents the positions from which the two objects can be discriminated by a passive agent (same data of Figure 3, top).

[Nolfi, Floreano, 2000][Nolfi, 2002 (Neurocomputing 42)]

Perceptual aliasing — summary

42

- In a nutshell
 - Memory-less controller assumes Markov property
 - Evolved trajectories can deal with perceptual aliasing
 - ▶ the robot experiences a sub-part of the sensorimotor space, where the Markov property always holds (or does not matter)

QUIZZ

QUIZZ

44

Q: si je ferme les yeux, est-ce que j'expérimente un alias perceptif si je tourne la tête?

1. oui
2. non

Q: un robot sans mémoire peut-il résoudre un problème dans un environnement non-Markovien

1. oui
2. non
3. ça dépend

Q: un robot sans mémoire résolvant une tâche dans un environnement non-Markovien...

1. n'aura jamais expérimenté des situations avec alias perceptif
2. pourra avoir expérimenté des situations avec alias perceptif

Q: plus l'environnement est riche, plus il risque d'être...

1. Markovien
2. non-Markovien

Q: nageant seul-e au milieu de l'océan sans terre en vue un jour d'orage à minuit, vous êtes...

1. dans un environnement Markovien
2. dans un environnement non-Markovien

Internal states

50

- Internal states
 - ▶ (1) increased computation power (2) capable of memory
 - ▶ Practical implementation (ex.) : recurrent neural networks

Introduced in the early 1930's by Tolman to demonstrate the latent learning of rats



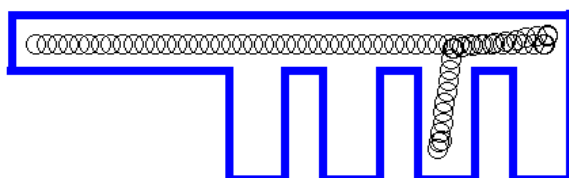
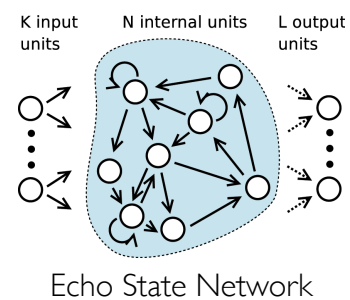
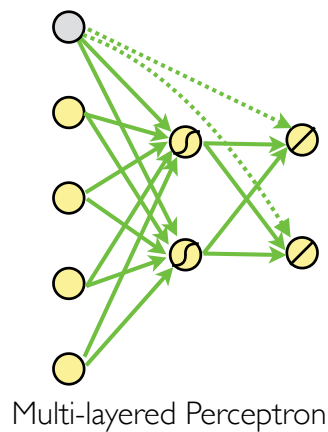
$$\mathcal{F}_0 = \sum_{t=0}^T ||x(t) - x^*||^2$$

$x(t)$: position of robot at time t

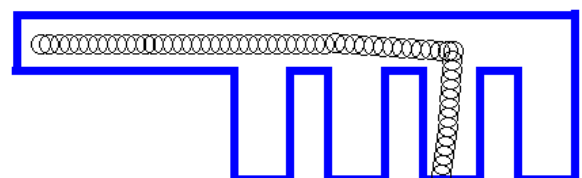
x^* : position of the target

Simulated Khepera II with 8 infrared sensors and 2 effectors (right and left wheels)

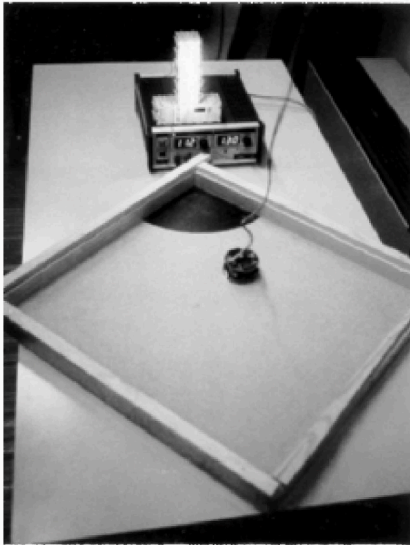
Echo state neural networks



evolving MLP
feed-forward networks



evolving ESN or NEAT
recurrent networks



Fitness

$$f(x) = \int_T V * (1 - sensor_{max})$$

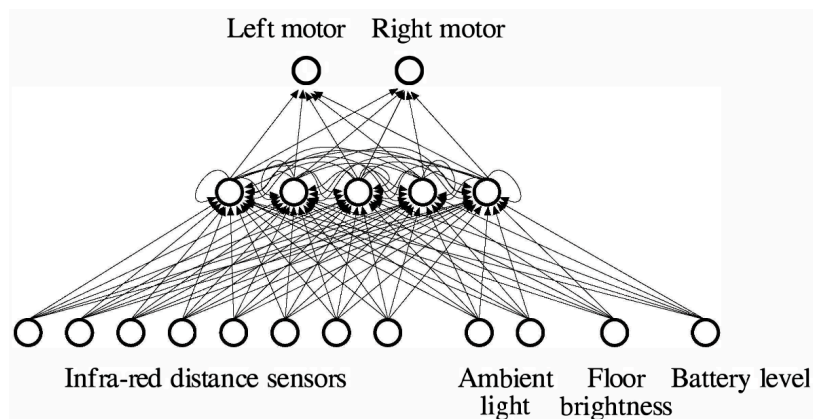
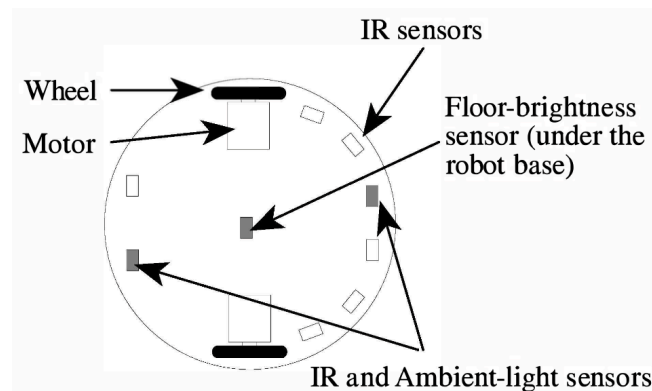
Hidden constraint

The robot must recharge when battery is low !

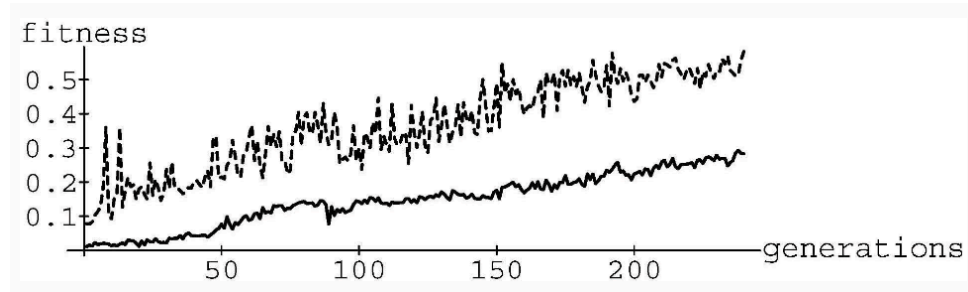
[nolfi, floreano, 2000]

Controller with internal states

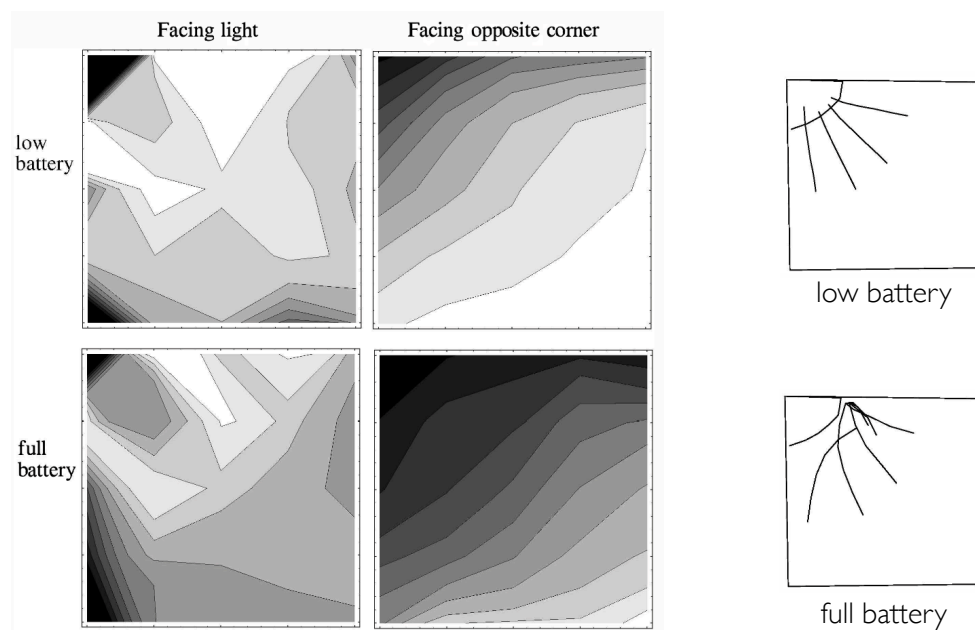
54

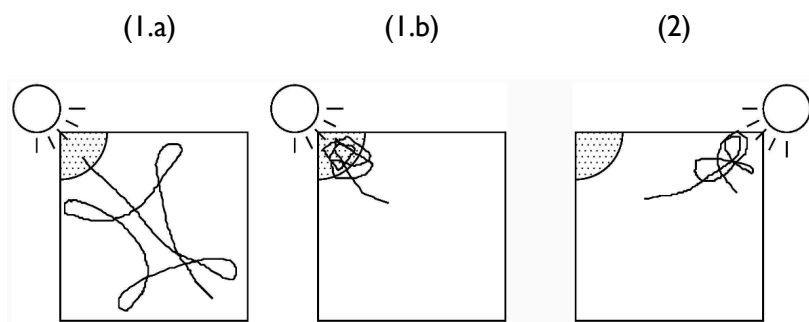


[nolfi, floreano, 2000]



(looks like canonical results for a not so difficult task)





Testing the best individual

Trajectory in the environment used for learning (1.a and 1.b)
and in a new environment with a different light source (2)

[nolfi, floreano, 2000]

Internal state — summary

58

- In a nutshell
 - Internal states can be used to map external structure
 - Internal states can be used to store information
 - Generalisation is constrained by the setup used

Levels of learning

- **Full general** : ad hoc controller (all controllers in previous slides)
 - ▶ the controller parameters are fixed, produce several behaviours
 - ▶ e.g. a controller can address several tasks at once
- **Plastic general**: adaptive controller (next slides)
 - ▶ the controller parameters change wrt the environment at hand
 - ▶ e.g. different tasks may require to change strategy completely

• Definitions

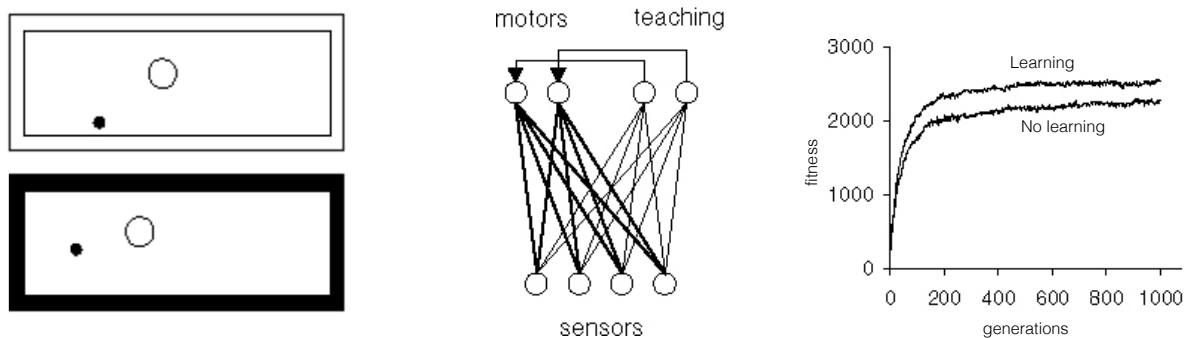
- evolutionary learning (in short: “evolution”)
 - ▶ adaptive process occurring in between evaluations
- individual learning (in short: “learning”)
 - ▶ adaptive process occurring during one evaluation

Both are adaptive processes working at different timescales

The shorter terms “evolution” and “learning” are used because of their meaning in biological systems

Combining learning and evolution — case study

62



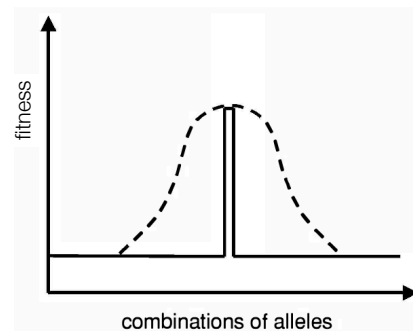
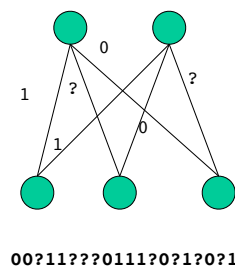
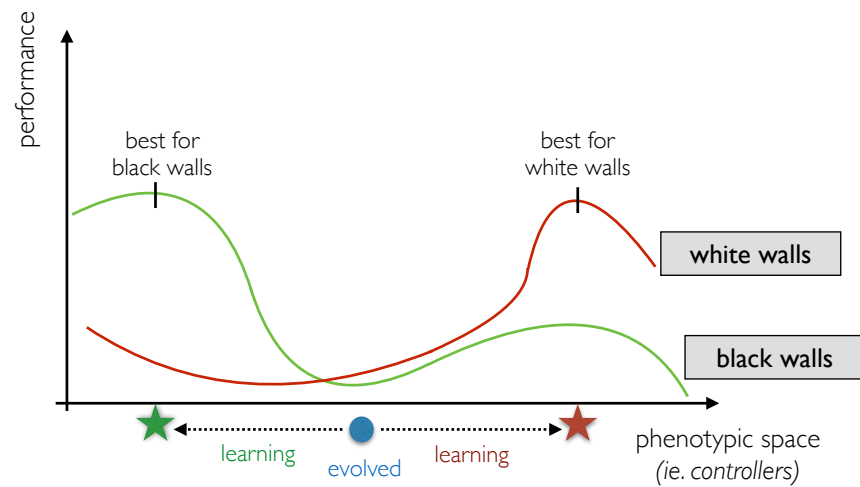
• Task

- ▶ The robot can experience one of two arenas, selection of the arena is random
- ▶ Wall colours change the sensory experience of the robot

• Auto-teaching architecture

- ▶ inputs: sensors —> outputs: actions *and* learning signal
- ▶ Perceptron: all parameters are evolved ; sensor-motor parameters are learnt
- ▶ Remark: learnt values of parameters are not copied back to genome

• Results : both faster and better results with learning



- Baldwin's effect with a simple model
 - ▶ Assume : learning task and evolutionary task are the same
 - ▶ Learning smoothes the fitness landscape
 1. Evolution provides a candidate solution
 2. Learning samples around the evolved solution

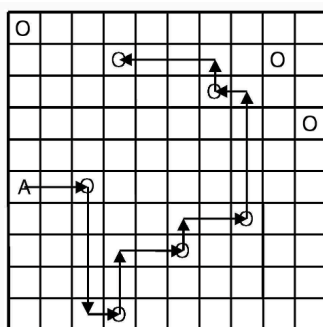
- Baldwin effect (1896)
 - ▶ “learning helps evolution”
 - learning smooth what evolution provides
 - ▶ “learning guides evolution”
 - **genetic assimilation** of previously learned traits may occur
 - rationale: innate trait directly available vs. time taken to learn

- Baldwinian vs Lamarckian evolution

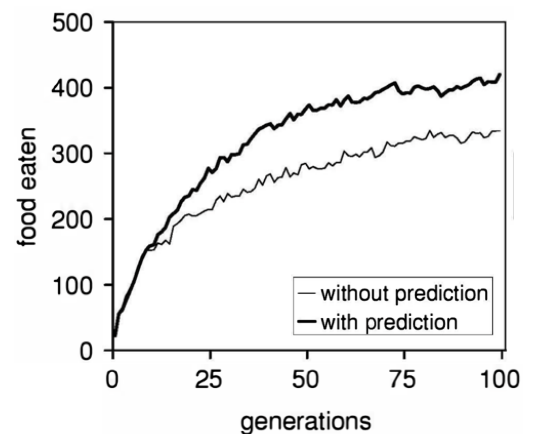
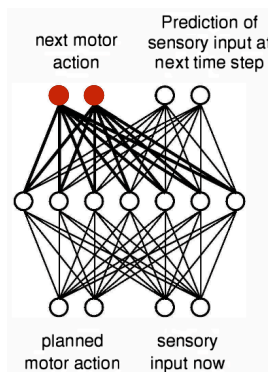
- Lamarckian: learnt parameters are *reintegrated* in the genome (not the case w/ Baldwinian)
- Good idea *iff* both tasks are aligned and static environment (Baldwinian is generally preferred)

Genetic assimilation

66



A : agent
O : “food” token (10x)



- Multi-layered Perceptron as a controller
 - ▶ inputs: (1) $sensors_t$ (2) $action_{t-1}$
 - ▶ outputs: (1) $action_t$ (2) **prediction of sensors at $t+1$**
- Levels of learning
 - ▶ evolutionary timescale: fitness is the number of tokens taken
 - ▶ learning timescale: minimise error in sensor predicted vs. true value
- Results : learning helps, then disappear as prediction error converge to zero!

- Individual learning vs. evolution
 - ▶ both are adaptive processes working at different timescales
- Transfer from one level to another
 - ▶ possible, but depends on the expressivity of the controller
- Open question
 - ▶ A full general controller...
 - can theoretically be a universal approximator*
 - but in practice it has a limited expressivity
 - ▶ A plastic general controller...
 - offers a path to problem decomposition
 - but requires a relevant learning signal

* or, with recurrent connections, equivalent to a Turing machine. At least theoretically.



QUIZZ

Q: l'assimilation génétique, c'est...

1. l'effet Waddington
2. l'effet Baldwin
3. l'effet Lamarck

Q: l'effet Baldwin c'est forcément l'assimilation génétique?

1. oui
2. non

Q: quand on regarde les patterns d'activation d'un réseau de neurones c'est...

1. absolument pas interprétable
2. complètement interprétable
3. un peu des deux

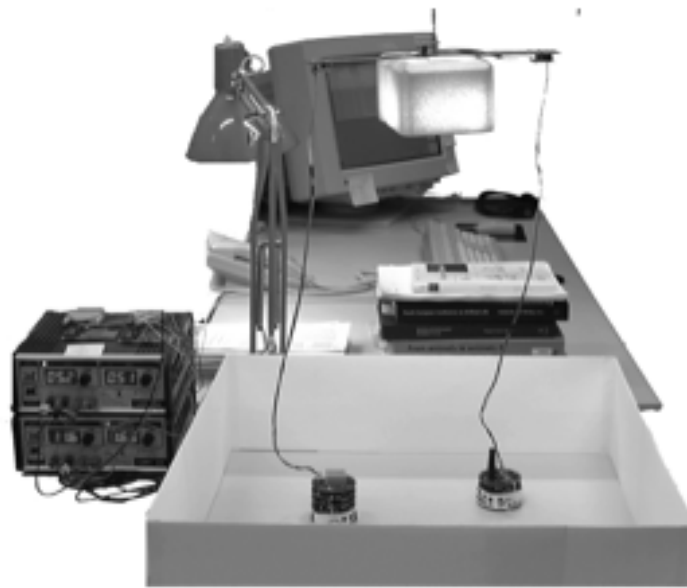
Q: dans l'architecture auto-teaching, le signal généré pour l'apprentissage donne...

1. les sorties moteur que le contrôleur aurait du produire
2. la prédiction de l'état des senseurs à $t+1$
3. je ne sais pas

- Q: dans l'architecture auto-teaching, que se passe t'il si on laisse converger l'apprentissage?
1. on garanti que la performance sera meilleure
 2. on en a aucune idée au delà du temps utilisés lors de l'évaluation pendant l'apprentissage

Reality gap

“Reality gap” is a term coined by Nigel Jakobi in 1995 [Jakobi, 1995, “Noise and the reality gap”]
It has been re-introduced as “Sim2real” in the Machine Learning field [Peng, 2018]



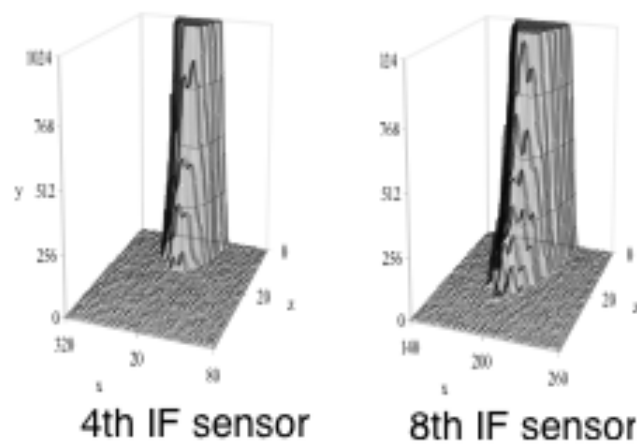
accurate but slow

[Nolfi and Floreano, 2000]

Reality gap — sensors

76

[Jakobi, 1995]



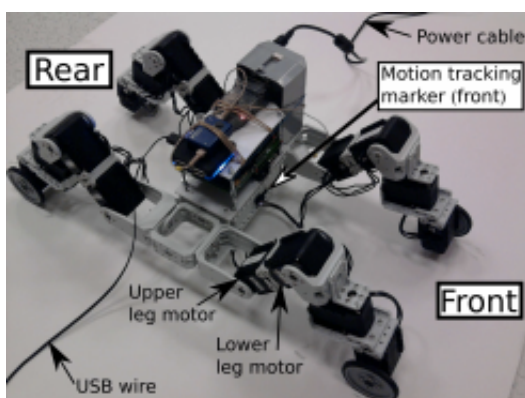
same condition, different sensors

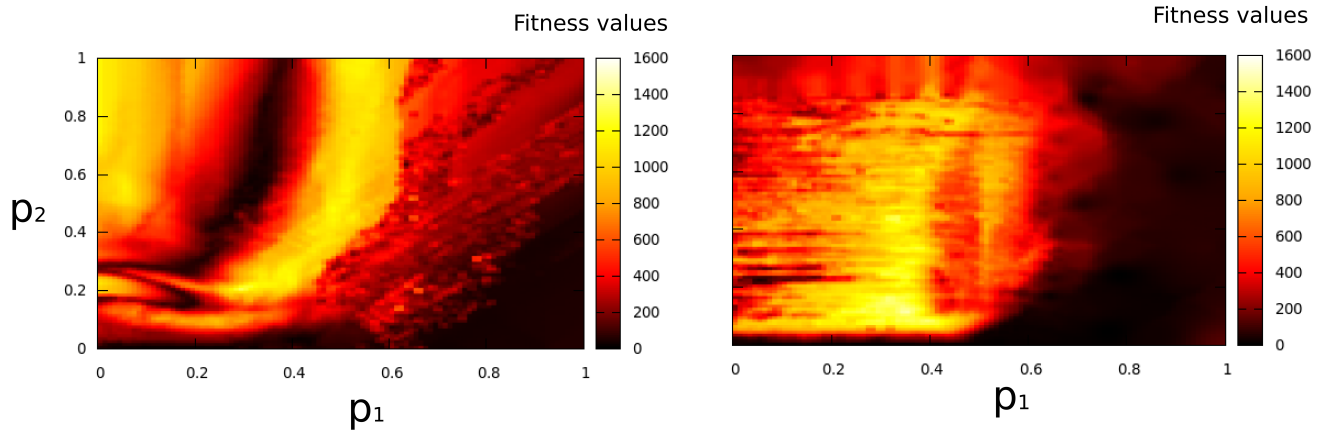
[Jakobi, 1995]



“full speed ahead!”

Example from [Hartland and Bredeche, 2006]





simulated robot

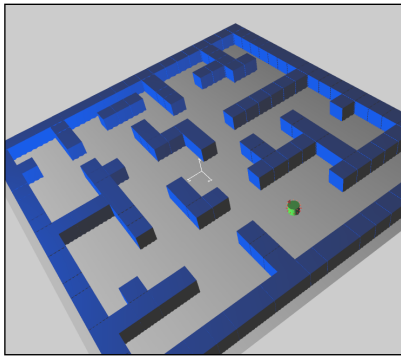
real robot

[Koos, Mouret, Doncieux, 2012]

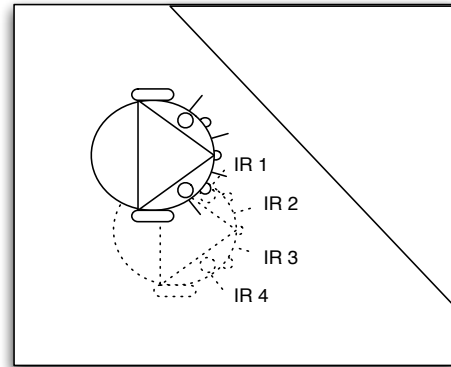
Addressing the reality gap

80

- Tradeoff
 - simulation is fast, but more or less inaccurate
 - reality is slow, but “true”
- Possible solutions
 - ...see next slides...



Simulation



Noise

here: "conservative noise"

how realistic is the added noise ?

sensors:

white noise

sampling real sensor noise [Miglino et al. 1997]

motors:

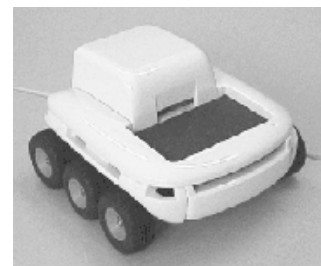
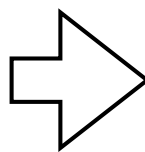
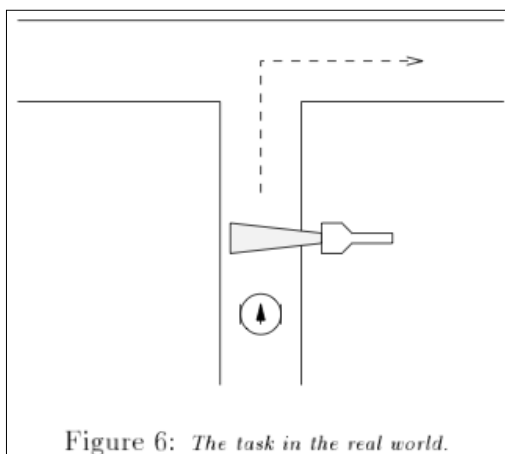
noise on commands

both:

conservative noise [Miglino et al. 1997]

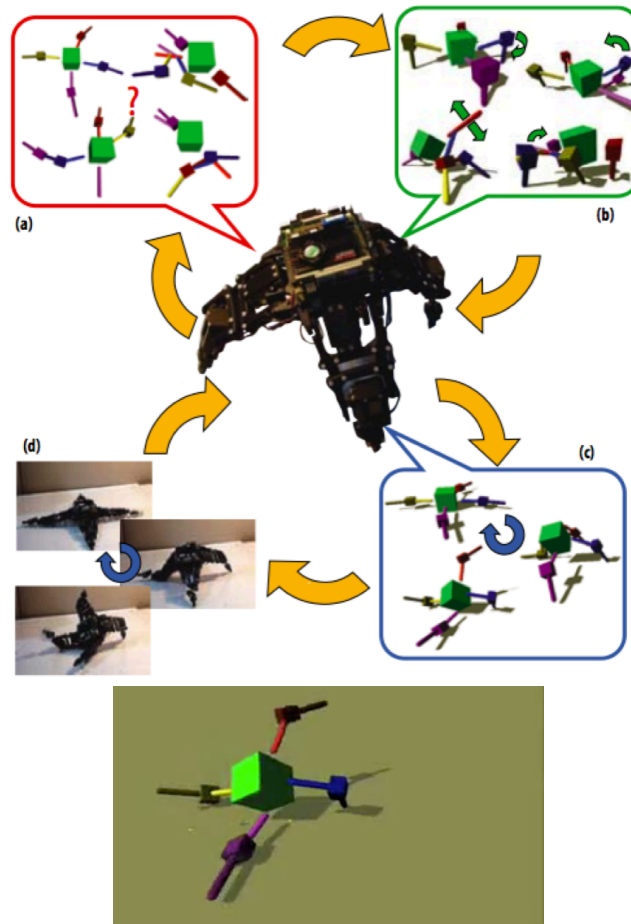
etc.

Using minimal simulation



problem decomposition

solving the task first, then solving the reality gap



[Bongard et al., 2006, 2009] — see also [Cully et al., 2015]

Reality gap — summary

- In a nutshell
 - simulation is useful
 - ▶ accelerate learning, accelerate implementation of ideas
 - ▶ but: what if fluid dynamics? complex physical interactions?
 - simulation may be inaccurate, favouring unrealistic behaviour
 - simulation may be too expensive (esp. w/ physics involved)
 - however: slow realistic simulation can avoid costly crashes...
 - real-world validation is mandatory
 - ▶ the reality gap is **too often ignored**...though it is **critical**
 - a vast amount of the literature only use simulation

Conclusions

86

- Summary
 - ▶ ER is a **blackbox** optimisation method for robotics
 - ▶ Opening the blackbox reveals **complex dynamics**
 - ▶ Not discussed here: ER works w/ large search space (same issues)
- Evolutionary Robotics and reinforcement learning
 - ▶ ER tackles similar problems as other RL methods
 - ▶ ER is a **direct policy search** method (cf. previous course & OS' courses)
- Open issues (see Stéphane Doncieux' courses)
 - ▶ **Performance-oriented fitness** can be misleading
 - ▶ Beyond search space R^n : **graphs** (NN topology, robot morphology)



REVIEW article

Front. Robot. AI, 03 March 2015

Sec. Robot Learning and Evolution

Volume 2 - 2015 | <https://doi.org/10.3389/frobt.2015.00004>

Evolutionary robotics: what, why, and where to

Stéphane Doncieux^{1,2*}Nicolas Bredeche^{1,2}Jean-Baptiste Mouret^{1,2}Agoston E. (Gusz) Eiben³¹ UMR 7222, ISIR, Sorbonne Universités, UPMC Univ Paris 06, Paris, France² UMR 7222, CNRS, ISIR, Paris, France³ Department of Computer Science, VU University, Amsterdam, Netherlands